

---

# AGENDA 7

---

## Estrutura de Decisão II

SEAD- Superintendência de Ensino e Pesquisa nas  
modalidades EaD e Aberta

GOVERNO DO ESTADO DE SÃO PAULO  
EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO  
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

DESENVOLVIMENTO DE SISTEMAS I

**Expediente**

Autores:

*PAULO EDUARDO CARDOSO ANDRADE*

*TATIANE TOLENTINO DE ASSIS*

*Revisão Técnica:*

*Kelly Cristiane de Oliveira Dal Pozzo*

São Paulo – SP, 2025



## MOMENTO REFLEXÃO



Na vida, raramente tomamos uma decisão baseada em apenas um motivo.

Quase sempre há mais de uma condição envolvida, pequenas variáveis que, somadas, definem o nosso caminho. Pense por um instante:

Você só sai de casa se não estiver chovendo, se tiver tempo livre e se estiver com disposição. Decide descansar se o dia foi cansativo ou se o corpo pede uma pausa.

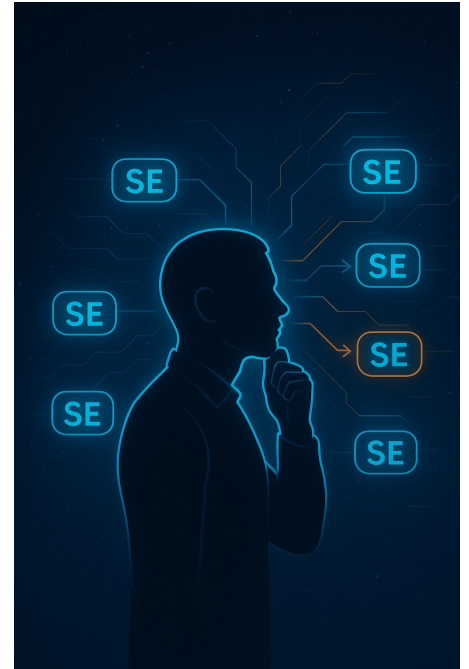
Resolve aceitar um convite se for com as pessoas certas e se o momento permitir.

As nossas escolhas são assim, um emaranhado de possibilidades, filtradas por combinações invisíveis de pensamentos. É como se cada decisão fosse um pequeno quebra-cabeça, onde cada peça importa: o tempo, o humor, a necessidade, a vontade.

**E é justamente nessas horas que percebemos: pensar é criar caminhos condicionais.**

Nem sempre o “sim” ou o “não” vem fácil; muitas vezes, ele depende de vários “se” acontecendo ao mesmo tempo.

Refletir sobre isso é reconhecer que nossas decisões são complexas e que compreender as relações entre condições é também compreender melhor o próprio ato de decidir.



## POR QUE APRENDER?



Se parar para pensar, o computador também toma decisões, mas de um jeito muito mais direto, frio e preciso.

Enquanto nós misturamos sentimentos, lembranças e intuições, ele trabalha apenas com condições bem definidas, que precisam ser claras e objetivas. Mas, no fundo, a lógica é parecida com a nossa.

Assim como nós avaliamos várias situações antes de agir, o computador também precisa combinar condições para chegar a uma resposta. Ele decide se algo deve acontecer ou não, dependendo de fatores que, juntos, formam um raciocínio completo.

**Aprender sobre as condições compostas e os operadores lógicos é aprender a traduzir o nosso modo de pensar para a linguagem da máquina.** É compreender que o

“se” humano, que depende de vários “e”, “ou” e “não” também existe dentro dos programas, só que expressos de outra forma.

Quando dominamos esse tipo de raciocínio, deixamos de apenas seguir instruções e passamos a criar sistemas que pensam. A cada nova condição que adicionamos, o computador se torna um pouco mais parecido com o modo como nós também decidimos, cruzando possibilidades, testando cenários e escolhendo o melhor caminho.

### PARA COMEÇAR O ASSUNTO

Depois de refletir sobre como tomamos decisões que envolvem várias condições ao mesmo tempo, é hora de olhar para o computador e perceber como ele faz algo muito parecido, só que de um jeito exato, matemático e sem dúvidas.

Enquanto nós pensamos em situações como “se eu tiver tempo e estiver com vontade, saio de casa”, o computador faz o mesmo tipo de raciocínio, só que usando palavras que ele entende: and, or e not.

Essas três palavras são os operadores lógicos, e com elas conseguimos fazer o computador combinar ideias e tomar decisões mais inteligentes.

É como se estivéssemos ensinando uma máquina a pensar de forma parecida com a nossa, mas com regras claras.



### MERGULHANDO NO TEMA

#### Operadores Lógicos

Os operadores lógicos são responsáveis pela elaboração de comparações especiais, possibilitando que uma única expressão de comparação receba mais de um operador relacional. Ele geralmente é utilizado em situações complexas, por exemplo:

Jovanir, um jovem de 20 anos, gostaria de saber se na próxima eleição ele será obrigado a votar ou se poderá votar de modo facultativo.

Para ser obrigado a votar, o eleitor deve ter a idade maior ou igual a dezoito anos e também o eleitor deve ter menos que 70 anos.

Neste caso, para que a expressão seja corretamente resolvida, necessitamos de um operador Lógico, que serão apresentados a seguir:

| Nome do Operador | Sintaxe (Pseudocódigo) | Sintaxe (Python) | Função                                                                      | Exemplo                                                        | Resultado  |
|------------------|------------------------|------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------|------------|
| <b>E (AND)</b>   | E                      | and              | O resultado será verdadeiro apenas se todas as condições forem verdadeiras. | (16 >= 16) E (16 < 18)<br>(16 >= 16) and (16 < 18)             | Verdadeiro |
| <b>OU (OR)</b>   | OU                     | or               | O resultado será verdadeiro se pelo menos uma condição for verdadeira.      | ((3 + 4) < 5) OU ((3 * 2) = 6) ((3 + 4) < 5) or ((3 * 2) == 6) | Verdadeiro |
| <b>NÃO (NOT)</b> | NÃO                    | not              | Inverte o valor lógico da expressão (verdadeiro $\leftrightarrow$ falso).   | NÃO (4 < 8) not (4 < 8)                                        | Falso      |

Observando a tabela dos Operadores Lógicos, podemos concluir que a expressão que resolveria o problema do Jovanir seria: ((20 >=18) && (20 < 70)).

Como a idade dele é maior ou igual a 18 e também é menor que 70, Jovanir é obrigado a votar.

### Tabela Verdade

Uma forma muito simples de lembrar qual é o operador correto para satisfazer uma expressão é utilizando a Tabela Verdade. Com ela podemos prever e entender melhor o funcionamento dos Operadores Lógicos.

A tabela consiste em separarmos a comparação em dois blocos, sendo o primeiro antes do Operador Lógico, e o segundo logo após o operador. Para simularmos os resultados, definimos os resultados como verdadeiro (V) ou falso (F), para facilitar a simulação e não perdemos tempo com a resolução das expressões.

Na tabela, verificamos todas as possibilidades, sendo ambas as comparações verdadeiras, ambas as comparações falsas ou apenas uma das comparações verdadeira, assim testamos as possibilidades que o operador pode proporcionar, como as tabelas a seguir nos mostram:

| Operador E (AND) |           |       |
|------------------|-----------|-------|
| Entrada 1        | Entrada 2 | Saída |
| V                | V         | V     |
| V                | F         | F     |
| F                | V         | F     |
| F                | F         | F     |

| Operador OU (OR) |           |       |
|------------------|-----------|-------|
| Entrada 1        | Entrada 2 | Saída |
| V                | V         | V     |
| V                | F         | V     |
| F                | V         | V     |
| F                | F         | F     |

\*Saída é o resultado das entradas de dados.

| Operador NÃO (NOT) |       |
|--------------------|-------|
| Entrada            | Saída |
| V                  | F     |
| F                  | V     |

## Operador and – (E)

O operador and exige que todas as condições sejam verdadeiras para que o resultado seja verdadeiro.

É o mesmo que dizer: “isso e aquilo precisam acontecer.”

| Operador | Expressão (Pseudocódigo) | Expressão (Python) | Resultado  |
|----------|--------------------------|--------------------|------------|
| E (AND)  | V E V                    | True and True      | Verdadeiro |
| E (AND)  | V E F                    | True and False     | Falso      |
| E (AND)  | F E V                    | False and True     | Falso      |
| E (AND)  | F E F                    | False and False    | Falso      |

Note que assim como na definição do operador E, o resultado da expressão foi verdadeiro, apenas quando ambos os lados da expressão são verdadeiros.

Imagine uma empresa está contratando e analisa currículo e precisa saber:

1. Candidato é maior de idade.
2. Candidato possui carteira de motorista.

Se ambas dessas condições for verdadeira, passa para próxima fase.

Veja o exemplo em Python:

python

```
idade = 20
carteira = True

if idade >= 18 and carteira:
    print("Próxima fase!")
else:
    print("recusado.")
```

A mensagem “Próxima fase” só aparece se as duas condições forem verdadeiras.

**Pensando como o computador:**

Quando escrevemos: `if condicao1 and condicao2`, o computador faz assim:

1. Ele avalia a `condicao1`.
2. Depois avalia a `condicao2`.
3. Só então decide se entra ou não no `if`.
4. Se qualquer uma delas for falsa, o resultado vira falso.

**É como uma corrente: se um elo arrebenta, a corrente não funciona.**

**Por que isso é importante?**

Para acessar um sistema, por exemplo, não basta ter o usuário certo, precisa ter usuário e senha.

Para fazer uma matrícula, às vezes precisa ter vaga e estar dentro do prazo.

Em controle de acesso físico, pode ser funcionário e estar no horário de expediente.

Em programação, o E (and) serve exatamente para isso: garantir que todas as regras de um processo estejam sendo atendidas.

**Dicas:**

Quando estiver escrevendo um `if` muito grande, pergunte para você mesmo:

“Eu quero que tudo isso seja verdade ao mesmo tempo?”

Se a resposta for sim, então o operador certo é o `and`.

**Operador or – OU**

O operador OU (`or`) é usado quando basta que uma das condições seja verdadeira para que o resultado também seja verdadeiro. Enquanto o E (`and`) é exigente (precisa de tudo verdadeiro) o OU (`or`) é flexível: aceita caminhos alternativos. Em outras palavras, ele permite que diferentes situações levem ao mesmo resultado.

É como dizer: “Se acontecer isso ou aquilo, tudo bem.”

| Operador       | Expressão (Pseudocódigo) | Expressão (Python) | Resultado  |
|----------------|--------------------------|--------------------|------------|
| <b>OU (OR)</b> | V OU V                   | True or True       | Verdadeiro |
| <b>OU (OR)</b> | V OU F                   | True or False      | Verdadeiro |
| <b>OU (OR)</b> | F OU V                   | False or True      | Verdadeiro |
| <b>OU (OR)</b> | F OU F                   | False or False     | Falso      |

Desta vez, apenas quando ambas as expressões são falsas o resultado foi falso.

Imagine uma empresa que libera o uso do estacionamento em duas situações:

1. O funcionário tem carro credenciado.
2. O funcionário é visitante autorizado.

Se qualquer uma dessas condições for verdadeira, a cancela abre.

Veja o exemplo em Python:

python

```
carro_credenciado = False
visitante_autorizado = True

if carro_credenciado or visitante_autorizado:
    print("Acesso liberado ao estacionamento!")
else:
    print("Acesso negado! Nenhum registro encontrado.")
```

Mesmo sem carro credenciado, o funcionário pôde entrar porque estava autorizado como visitante.

### Pensando como o computador

Ao ler um if com or, o computador verifica as condições da esquerda para a direita.

1. Assim que encontra uma verdadeira, ele já executa o bloco de código, nem precisa conferir as outras.
2. Isso torna o or eficiente em casos onde queremos dizer:

**“Se pelo menos uma coisa for verdadeira, pode seguir em frente.”**

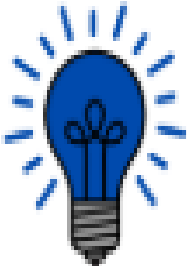
**Por que isso é importante?**

Ele serve para representar decisões flexíveis, nas quais mais de um cenário é aceitável.

Por exemplo:

- O sistema envia notificação se o prazo estiver perto do fim ou já tiver expirado.
- Um login pode ser feito com e-mail ou CPF.
- Uma promoção vale se o cliente for aluno ou ex-aluno.

Assim, o OR ajuda o programa a tomar decisões quando existem várias condições possíveis para a mesma ação.

**Dicas:**

Quando estiver escrevendo um if com várias condições, pergunte a si mesmo:

“Basta uma dessas condições ser verdadeira para que isso aconteça?”

Se a resposta for sim, então o operador certo é o or.

Use o or sempre que existirem caminhos alternativos para o mesmo resultado — ou seja, quando qualquer uma das situações for suficiente para que o programa execute a ação.

**Operador not – Não**

O operador not é o mais simples dos três, mas também um dos mais importantes. Ele é responsável por inverter o valor lógico de uma condição, ou seja, transforma o que é verdadeiro em falso, e o que é falso em verdadeiro. Enquanto o and e o or servem para combinar condições, o not serve para negar uma ideia.

É como dizer para o computador:

“Faça isso, a menos que a condição seja verdadeira.”

| Operador  | Expressão (Pseudocódigo) | Expressão (Python) | Resultado  |
|-----------|--------------------------|--------------------|------------|
| NÃO (NOT) | NÃO V                    | not True           | Falso      |
| NÃO (NOT) | NÃO F                    | not False          | Verdadeiro |

Para o operador NÃO, o resultado da expressão é apenas invertido.

Imagine uma empresa onde o sistema precisa liberar o uso da impressora apenas para quem não estiver em manutenção.

Veja o exemplo em Python:

python

```
em_manutencao = False

if not em_manutencao:
    print("Impressora disponível para uso.")
else:
    print("Impressora em manutenção.")
```

Observe que o not inverte o valor da variável em\_manutencao é False, mas o not em\_manutencao vira True, e, portanto, o sistema libera o uso.

### Pensando como o computador

Quando o Python lê um not, ele faz o seguinte raciocínio:

1. “Pegue o resultado dessa condição e troque pelo oposto.”
2. Isso torna o código mais limpo e direto, principalmente em situações que envolvem negação lógica.
3. Em vez de escrever algo como: if em\_manutencao == False:
4. Podemos simplesmente escrever: if not em\_manutencao:

O resultado é o mesmo, mas o código fica mais legível e profissional.

#### Por que isso é importante?

O not é um operador unário, ou seja, ele atua sobre uma única condição.

Ele é usado quando queremos que uma ação aconteça justamente quando algo não for verdadeiro.

Por exemplo:

- Mostrar um aviso somente quando o usuário não estiver logado.
- Enviar alerta se o equipamento não estiver online.
- Liberar acesso quando o funcionário não estiver suspenso.

O not ajuda o programa a pensar pelo lado oposto — a enxergar o que não está acontecendo, e reagir a isso.

**Dicas:**

Quando estiver escrevendo um if e pensar:

“Quero que isso aconteça somente se essa condição não for verdadeira...” então o operador certo é o not.

Use o not sempre que precisar negar uma ideia, impedir uma ação ou inverter o sentido lógico de uma verificação.

## Um olhar à frente – outros operadores que encontraremos

Até agora, aprendemos a tomar decisões com and, or e not, operadores que ajudam a combinar e negar condições, esses operadores são comuns na lógica de programação. Mas o Python oferece outros recursos que veremos mais adiante, operadores que ampliam a forma de comparar, verificar e analisar informações.

Esses operadores são muito úteis quando trabalhamos com listas, textos, variáveis nulas ou objetos, e fazem parte do raciocínio lógico de programas mais avançados.

| Operador  | Significado    | Exemplo                  | Função principal                                            |
|-----------|----------------|--------------------------|-------------------------------------------------------------|
| <b>in</b> | Pertence a     | uf in ["SP", "RJ", "MG"] | Verifica se um valor está presente em uma coleção ou texto. |
| <b>is</b> | É (identidade) | x is None                | Verifica se duas variáveis são o mesmo objeto na memória.   |

Em breve, ao estudar listas, você verá como in e is se tornam ferramentas indispensáveis para buscar, validar e comparar dados com elegância e precisão.

## VOCÊ NO COMANDO

Você está desenvolvendo o sistema de controle de acesso da empresa.

Esse sistema deve verificar três aspectos principais antes de liberar o login de um funcionário:

1. O sistema não pode estar em manutenção.
2. O funcionário precisa estar ativo.
3. O acesso é permitido se o funcionário tiver autorização especial ou for sexta-feira (dia de home office).

Antes de juntar tudo, vamos entender como cada operador funciona separadamente.

## Testando o operador NOT

O operador NOT serve para negar uma condição. Queremos permitir o acesso somente se o sistema não estiver em manutenção.

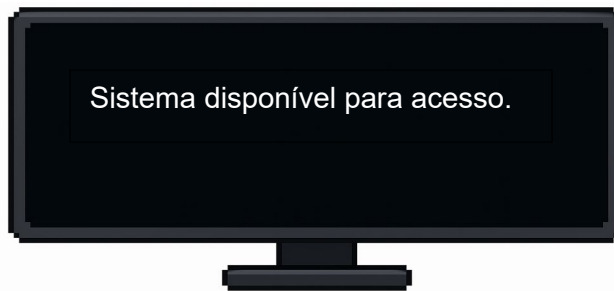
python

```
sistema_em_manutencao = False

if not sistema_em_manutencao:
    print("Sistema disponível para acesso.")
else:
    print("Sistema em manutenção. Acesso bloqueado.")
```

Quando sistema\_em\_manutencao for False, o NOT o transforma em True, e o acesso é liberado.

Saída:



## Testando o operador and

O operador and exige que todas as condições sejam verdadeiras para executar o bloco. Neste caso desejamos liberar o acesso somente se o usuário estiver ativo e o sistema não estiver em manutenção.

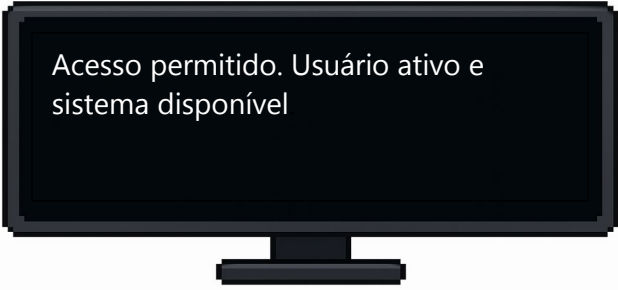
python

```
usuario_ativo = True
sistema_em_manutencao = False

if usuario_ativo and not sistema_em_manutencao:
    print("Acesso permitido. Usuário ativo e sistema disponível.")
else:
    print("Acesso negado! Verifique as condições.")
```

Se qualquer uma das condições for falsa, o acesso não será liberado.

Veja o resultado:



Acesso permitido. Usuário ativo e sistema disponível

## Testando o operador OR

O operador `or` permite que basta uma condição ser verdadeira para o resultado ser verdadeiro. O funcionário pode acessar o sistema se tiver autorização especial ou se for sexta-feira.

python

```
autorizacao_especial = False
dia_semana = "Sexta"

if autorizacao_especial or dia_semana == "Sexta":
    print("Acesso liberado: condição alternativa válida.")
else:
    print("Acesso negado! Nenhuma condição atendida.")
```

Mesmo sem autorização, o acesso é liberado porque é sexta-feira.



Sistema disponível para acesso.

## E se juntarmos tudo?

Quando unimos todas as partes — o `not`, o `and` e o `or` — criamos algo mais poderoso: um encadeamento lógico, capaz de tomar decisões completas e realistas, como um sistema de verdade.

Cada operador faz o seu papel dentro desse fluxo:

- O `not` filtra o que não pode acontecer.
- O `and` garante que tudo o que é obrigatório seja atendido.
- O `or` abre espaço para alternativas válidas.

Juntos, eles constroem uma linha de raciocínio encadeada, na qual uma decisão depende da outra, até chegar ao resultado.

Veja o código Python:

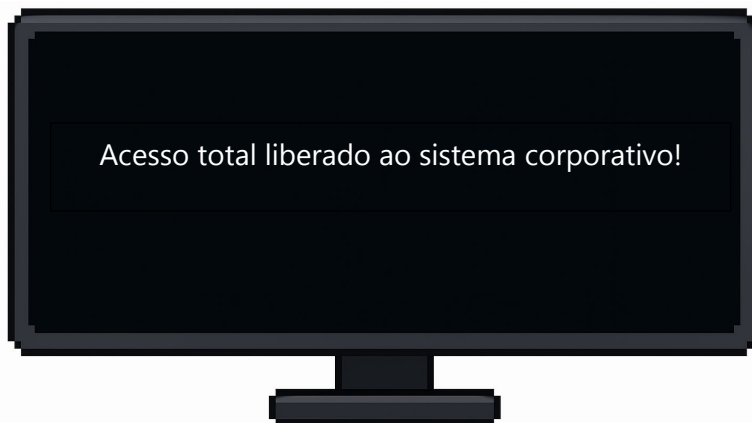
python

```
usuario_ativo = True
sistema_em_manutencao = False
autorizacao_especial = False
dia_semana = "Sexta"

if usuario_ativo and not sistema_em_manutencao and (autorizacao_especial or dia_semana == "Sexta"):
    print("Acesso total liberado ao sistema corporativo!")
else:
```

Nesse ponto, o programa avalia passo a passo:

1. Se o sistema não está em manutenção;
2. Se o usuário está ativo;
3. E, por fim, se existe uma forma de liberar o acesso (autorização especial ou sexta-feira).



**O raciocínio segue um “encadeamento” natural, quase como uma conversa entre as condições.**

**É o mesmo tipo de lógica que usamos no dia a dia, só que agora traduzida para o Python.**

Assim nasce o verdadeiro poder dos operadores lógicos: ligar ideias diferentes para formar um pensamento coerente. Eles transformam uma simples verificação em um processo de decisão completo, capaz de avaliar cenários e reagir de forma inteligente.



## CALMA

Pode parecer que juntar todas essas condições de uma vez é complicado, e é mesmo, no começo. Mas não se preocupe: esse tipo de raciocínio se constrói aos poucos, com treino, repetição e paciência.

Ninguém nasce sabendo encadear condições complexas; é algo que o cérebro aprende conforme você experimenta os operadores de forma separada e observa o comportamento de cada um.

Primeiro, entendemos o que o `not` faz, depois o `and`, depois o `or`. Só então começamos a perceber que eles podem conversar entre si, formando blocos de lógica mais elaborados.

Pense nisso como montar um quebra-cabeça:

cada operador é uma peça diferente, e você precisa conhecer o formato de cada uma antes de saber onde elas se encaixam.

No início, o processo é lento, mas com o tempo você vai enxergando padrões, e, sem perceber, começa a construir raciocínios completos com naturalidade.

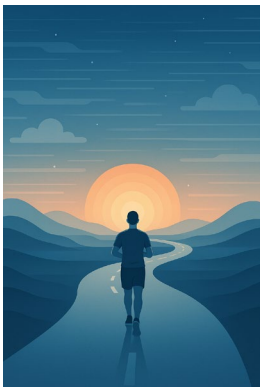
### O segredo é não ter pressa.

Faça testes, mude valores, leia o código em voz alta, tente prever o resultado antes de rodar. Quanto mais você praticar, mais natural será “pensar como o computador”, sem precisar forçar a mente para entender cada detalhe.

Programar é isso: treinar o raciocínio até ele se tornar parte de você. E quando isso acontece, você não apenas entende o que o código faz, você passa a antecipar o que ele vai fazer, porque seu pensamento já está estruturado em lógica.

Então, vá com calma.

Use os operadores separados, brinque com eles, observe os resultados. Logo, quando perceber, estará criando condições complexas e elegantes sem esforço algum, porque o raciocínio lógico, como qualquer habilidade, floresce com a prática e a paciência.



**Programar é uma maratona, não uma corrida de 100 metros.** Cada linha de código é um passo, e o importante não é chegar rápido, mas aprender a continuar com constância e clareza.

## Conhecendo o match (ou switch)



Depois de dominar as estruturas condicionais com `if`, `elif` e `else`, e entender como combinar ideias com os operadores lógicos, você está pronto para conhecer uma nova forma de tomar decisões no Python: o `match`. Introduzido a partir da versão Python 3.10, o `match` é parecido com o famoso `switch` de outras linguagens, como C ou JavaScript.

Ele serve para comparar um mesmo valor com várias possibilidades, tornando o código mais limpo, organizado e fácil de ler.

**Por que usar o match?**

Quando temos muitas opções dentro de um mesmo `if` ou uma sequência de `elif`, o código pode ficar longo e cansativo. O `match` veio para simplificar isso, ajudando o Python a decidir entre vários caminhos com elegância.

**Exemplo prático**

Imagine um sistema simples que mostra a mensagem de acordo com o dia da semana digitado pelo usuário:

```
python
dia = input("Digite o dia da semana: ")

match dia:
    case "Segunda":
        print("Início de semana! Foco total.")
    case "Sexta":
        print("Sextou! Energia renovada.")
    case "Sábado" | "Domingo":
        print("Final de semana! Hora de descansar.")
    case _:
        print("Dia comum.")
```

**Compreendendo o código:**

O comando `match` verifica o valor da variável (neste caso, `dia`).

Cada `case` representa uma possibilidade.

- O símbolo `|` significa “ou”, permitindo testar múltiplos valores na mesma linha.
- O `_` (underline) é o coringa ele representa qualquer outro valor não listado acima.

**Por que isso é importante?**

O match deixa o código mais parecido com uma tabela de decisões, ideal para situações como:

- Menus de opções;
- Escolha de comandos;
- Identificação de categorias;
- Processos automáticos baseados em um valor principal.



**O match não substitui o if, mas complementa, é outra ferramenta para o mesmo raciocínio lógico que você já está desenvolvendo.**

**Vamos programar!****Menu do Laboratório com match**

Você vai criar um pequeno sistema de menu para o laboratório de informática. O usuário digita uma opção e o programa responde o que foi escolhido.

As opções serão:

- 1 – Listar computadores disponíveis
- 2 – Abrir chamado de manutenção
- 3 – Ver regras de uso do laboratório

Qualquer outra tecla “Opção inválida”

Faça isso usando o comando match do Python.

Veja o código Python:

python

```
opcao = input("Digite uma opção (1 - PCs, 2 - Manutenção, 3 - Regras): ")

match opcao:
    case "1":
        print("Computadores disponíveis: Lab01-PC01 ao Lab01-PC20.")
    case "2":
        print("Abrindo chamado de manutenção... descreva o problema.")
    case "3":
        print("Regras: uso individual, silêncio e não alterar configurações.")
    case _:
        print("Opção inválida. Tente novamente.")
```

- O match está olhando sempre para a mesma variável (opcao);
- Cada case é um caminho possível;
- O case `_`: é o “senão” do match;

### Menu do Laboratório com múltiplas opções

Agora, vamos deixar o menu mais inteligente:

- alguns usuários podem digitar a opção em número (ex.: 1) e outros podem digitar em texto (ex.: "um").

O programa deve entender as duas formas.

### Veja o código Python

python

```
opcao = input("Digite uma opção (1 - PCs, 2 - Manutenção, 3 - Regras): ")

match opcao:
    case "1" | "um" | "Um":
        print("Computadores disponíveis: Lab01-PC01 ao Lab01-PC20.")
    case "2" | "dois" | "Dois":
        print("Abrindo chamado de manutenção... descreva o problema.")
    case "3" | "tres" | "Três" | "três":
        print("Regras: uso individual, silêncio e não alterar configurações.")
    case _:
        print("Opção inválida. Tente novamente.")
```



O símbolo `|` significa “ou”, funcionando como uma forma alternativa de escrever várias opções que levam ao mesmo resultado. Isso deixa o código mais inclusivo e flexível, o programa entende diferentes formatos da mesma ideia.

Perceba como o match permite organizar decisões complexas de forma limpa. Sem precisar de vários `elif`, você agrupa valores parecidos no mesmo bloco, tornando o código fácil de ler, manter e evoluir.

É como dizer ao Python?

“Se o usuário digitar **1 ou um ou Um**, quero que aconteça a mesma coisa.”

Essa é a beleza do match: ele não substitui o `if`, mas amplia seus horizontes, tornando a lógica mais natural e expressiva.



Assista a aula explicativa:



Disponível em: <https://www.youtube.com/watch?v=-5usX7zMwY8>  
Acessado em 27/03/2026.